X

(https://swayam.gov.in) (https://swayam.gov.in/nc_details/NPTEL)

pradipmaity9907@gmail.com

NPTEL (https://swayam.gov.in/explorer?ncCode=NPTEL) » The Joy of Computing using Python (course)

Course outline

About NPTEL ()

How does an NPTEL online course work? ()

Week 1 ()

Week 2 ()

Week 3 ()

week 4 ()

Week 5 ()

- Introduction to Dictionaries (unit?unit=104&lesson=105)
- Speech to Text : No need to write 01 (unit?unit=104&lesson=106)

Week 5: Assignment 5

The due date for submitting this assignment has passed.

Due on 2026-02-25, 23:59 IST.

Assignment submitted on 2026-02-24, 23:35 IST

Dilrez Builds a Smart Student Record System

Dilrez is a teaching assistant who is asked to create a **Smart Student Record System** for a small institute.
The system must store, update, analyze, and report student data efficiently.

To achieve this, Dilrez chooses **Python Dictionaries** because:

- Each student has a **unique roll number**
- Each roll number maps to **multiple details**
- Data needs to be **updated, searched, and processed quickly**

What Is a Dictionary? (Key–Value Mapping)

A dictionary stores data in **key : value** form.

```
student = {
    "roll": 101,
    "name": "Ayaan",
    "age": 20
}
```

2. Accessing Dictionary Data

```
print(student["name"])
print(student.get("age"))
```

- Speech to Text : No need to write 02 (unit?unit=104&lesson=107)
- Speech to Text : No need to write 03 (unit?unit=104&lesson=108)
- Monte Hall : 3 doors and a twist 01 (unit?unit=104&lesson=109)
- Monte Hall : 3 doors and a twist 02 (unit?unit=104&lesson=110)
- Rock, Paper and Scissor : Cheating not allowed !! 01 (unit?unit=104&lesson=111)
- Rock, Paper and Scissor : Cheating not allowed !! 02 (unit?unit=104&lesson=112)
- Rock, Paper and Scissor : Cheating not allowed !! 03 (unit?unit=104&lesson=113)
- Rock, Paper and Scissor : Cheating not allowed !! 04 (unit?unit=104&lesson=114)
- Sorting and Searching : 20 questions

.get() is safer because it does not cause an error if the key is missing.

3. Adding & Updating Data in a Dictionary

Dilrez now wants to add marks.

```
student["marks"] = 85
student["age"] = 21
```

4. Dictionary of Lists (One Student, Multiple Scores)

Each student has marks in multiple tests.

```
student = {
    "roll": 101,
    "name": "Ayaan",
    "marks": [78, 85, 90]
}
```

Processing the list inside a dictionary

```
average = sum(student["marks"]) / len(student["marks"])
print(average)
```

5. Dictionary of Dictionaries (Structured Records)

Now Dilrez stores **multiple students**.

```
students = {
    101: {"name": "Ayaan", "marks": [78, 85, 90]},
    102: {"name": "Riya", "marks": [88, 92, 79]},
    103: {"name": "Kabir", "marks": [70, 75, 80]}
}
```

Accessing nested data

```
print(students[102]["name"])
print(students[101]["marks"][1])
```

This structure is ideal for **databases, JSON, APIs**.

6. List of Dictionaries (Row-Wise Data)

Sometimes data comes row-by-row (like Excel).

game 01 (unit?unit=104&lesson=115)
- Sorting and Searching : 20 questions game 02 (unit?unit=104&lesson=116)
- Sorting and Searching : 20 questions game 03 (unit?unit=104&lesson=117)
- Sorting and Searching : 20 questions game 04 (unit?unit=104&lesson=118)
- Sorting and Searching : 20 questions game 05 (unit?unit=104&lesson=119)
- Sorting and Searching : 20 questions game 06 (unit?unit=104&lesson=120)
- Sorting and Searching : 20 questions game 07 (unit?unit=104&lesson=121)
- Sorting and Searching : 20 questions game 08 (unit?unit=104&lesson=122)
- Week 5 Feedback Form: The Joy of Computing using Python (unit?

```
students = [
    {"roll": 101, "name": "Ayaan", "marks": 85},
    {"roll": 102, "name": "Riya", "marks": 90},
    {"roll": 103, "name": "Kabir", "marks": 78}
]
```

Searching in list of dictionaries

```
for student in students:
    if student["roll"] == 102:
        print(student["name"])
```

7. Looping Through Dictionaries

Keys only

```
for key in student:
    print(key)
```

Values only

```
for value in student.values():
    print(value)
```

Key–Value pairs

```
for key, value in student.items():
    print(key, value)
```

Complete Mini Program

unit=104&lesson=123)

- Week 5 : Programming Assignment 1 (/noc26_cs84/progassignment?name=623)
- Week 5 : Programming Assignment 2 (/noc26_cs84/progassignment?name=624)
- Week 5 : Programming Assignment 3 (/noc26_cs84/progassignment?name=625)
- **Quiz: Week 5: Assignment 5 (assessment?name=632)**

Week 6 ()

Week 7 ()

Week 8 ()

Week 9 ()

Week 10 ()

Week 11 ()

Week 12 ()

Text Transcripts ()

Download Videos ()

```
students = {
    101: {"name": "Ayaan", "marks": [78, 85, 90]},
    102: {"name": "Riya", "marks": [88, 92, 79]}
}

def class_average(students_dict):
    total = 0
    count = 0
    for student in students_dict.values():
        total += sum(student["marks"])
        count += len(student["marks"])
    return total / count

print(class_average(students))
```

1) Dilrez wants to extract the first test score of Riya from a nested student record. What will be printed? ***1 point***

```
students = {
    101: {"name": "Ayaan", "marks": [78, 85, 90]},
    102: {"name": "Riya", "marks": [88, 92, 79]}
}

print(students[102]["marks"][0])
```

- 92
- 79
- 88
- Error

Yes, the answer is correct.
Score: 1

Accepted Answers:
88

2) A new test score is added for Kabir. How many marks does Kabir finally have? ***1 point***

```
student = {
    "name": "Kabir",
    "marks": [70, 75, 80]
}

student["marks"].append(85)
print(len(student["marks"]))
```

- 3
- 85
- 4
- Error

Yes, the answer is correct.

Score: 1

Accepted Answers:

4

3) Dilrez modifies a list stored inside a dictionary and then reuses the same reference. What is printed? ***1 point***

```
data = {"nums": [1, 2, 3]}
ref = data["nums"]

ref.append(4)
print(data["nums"])
```

- [1, 2, 3]
- [1, 2, 3, 4]
- Error
- [4]

Yes, the answer is correct.
Score: 1

Accepted Answers:

[1, 2, 3, 4]

4) Dilrez slices a list obtained from a dictionary and modifies the slice.What happens to the original list? ***1 point***

```
student = {"marks": [10, 20, 30, 40]}
subset = student["marks"][:2]
subset.append(99)

print(student["marks"])
```

- [10, 20, 30, 40]
- [10, 20, 99]
- Error
- [10, 20]

Yes, the answer is correct.
Score: 1

Accepted Answers:

[10, 20, 30, 40]

5) A function reassigns a dictionary parameter instead of mutating it. What value is printed? ***1 point***

```
def update(d):
    d = {"x": 100}

data = {"x": 10}
update(data)
print(data["x"])
```

- (selected) 10
- 100
- None
- Error

Yes, the answer is correct.
Score: 1

Accepted Answers:
10

6) Dilrez modifies a dictionary while iterating over its keys *without creating a copy*. What happens when the code runs? ***1 point***

```
data = {"a": 1, "b": 2}

for k in data.keys():
    if k == "a":
        data["c"] = 3

print(len(data))
```

- 2
- 3
- 1
- (selected) Runtime Error

Yes, the answer is correct.
Score: 1

Accepted Answers:
Runtime Error

7) Dilrez draws a bar graph to compare students fairly, even though they have attempted a different number of tests. Which student's bar will be the tallest? ***1 point***

```
students = {
    "Ayaan": [70, 80, 90],
    "Riya": [85, 90],
    "Kabir": [60, 65, 70, 75]
}
```

- Kabir
- Ayaan
- (selected) Riya
- All bars will be equal

Yes, the answer is correct.
Score: 1

Accepted Answers:
Riya

8) Dilrez plots a line graph showing performance of each test across all students. Which test appears as the highest point? ***1 point***

```
students = {
    1: {"marks": [50, 60, 70]},
    2: {"marks": [60, 70, 80]},
    3: {"marks": [70, 80, 90]}
}
```

- Test 2
- Test 1
- Test 3
- All tests overlap

Yes, the answer is correct.
Score: 1

Accepted Answers:
Test 3

9) Dilrez creates a pie chart showing pass vs fail distribution (pass ≥ 60). Which section occupies more area? ***1 point***

```
records = [
    {"name": "A", "marks": 45},
    {"name": "B", "marks": 75},
    {"name": "C", "marks": 65},
    {"name": "D", "marks": 55},
    {"name": "E", "marks": 85}
]
```

- Equal distribution
- Pass section
- Cannot be inferred
- Fail section

Yes, the answer is correct.
Score: 1

Accepted Answers:
Pass section

10) Dilrez plots a cumulative trend graph of weekly progress. What overall shape will the graph show? ***1 point***

```
data = {
    "week1": [10, 20],
    "week2": [30],
    "week3": [40, 50]
}
```

- Decreasing trend

- Flat line
- Random fluctuations
- Increasing trend

Yes, the answer is correct.
Score: 1

Accepted Answers:
Increasing trend

Searching for a Book ID in a Digital Library

A college library maintains a **sorted list of book IDs**.
Each time a student requests a book, the system must **search for the book ID** and also **measure how many checks (iterations)** were needed to find it.

Two interns, **Aarav and Neel**, suggest two different approaches:

- Aarav uses **Linear Search**
- Neel uses **Binary Search**

The librarian wants to understand:
1. How each method works
2. How many iterations each method takes
3. Which method is more efficient and *why*

Part 1: Linear Search Case Study

Idea Behind Linear Search

Linear search works exactly like **checking books one by one on a shelf**, starting from the first book until the required one is found.

- No assumptions about sorting
- Simple and intuitive
- Works for **any list**

```
def linear_search(a, x):
    count = 0

    for i in range(len(a)):
        count += 1
        if a[i] == x:
            print(f"Element found at position {i}.")
            return count

    print("Number is not present.")
    return count
```

Sample Scenario

a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

x = 7

We are searching for the element **7** in the list:

The algorithm starts from the **first index (0)** and moves forward **one element at a time.**

1. **First iteration**
The element at index 0 is 1.
It does not match 7, so the counter increases and the search continues.

2. **Second iteration**
The element at index 1 is 2.
Still not a match.The algorithm moves ahead.

3. **Third iteration**
The element at index 2 is 3.
Again, no match.

4. **Fourth iteration**
The element at index 3 is 4.
The search continues.

5. **Fifth iteration**
The element at index 4 is 5.
No match yet. 6.

6. **Sixth iteration**
The element at index 5 is 6.
The algorithm proceeds further.

7. **Seventh iteration**
The element at index 6 is 7.
This time, the element matches the target.

Why the iteration count is 7
The counter increases **before checking each element**, and the algorithm must **visit every position from index 0 to index 6.**
So, even though the element is at position 6, it takes **7 checks** to find it.

Part 2: Binary Search Case Study

Idea Behind Binary Search

Binary search is like **opening a dictionary:**

- You don't start from page 1
- You open the **middle**, then decide left or right

Important Requirement:
The list **must be sorted**

```python
def binary_search(a, x):
    first_pos = 0
    last_pos = len(a) - 1
    flag = 0
    count = 0

    while first_pos <= last_pos and flag == 0:
        count += 1
        mid = (first_pos + last_pos) // 2

        if x == a[mid]:
            flag = 1
            print(f"Element found at position {mid}.")
            return count
        elif x < a[mid]:
            last_pos = mid - 1
        else:
            first_pos = mid + 1

    print("Number is not present.")
    return count
```

Same Scenario, Different Strategy

a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

x = 7

Binary search works by **eliminating half of the list** after every comparison.

1. **First iteration**
The middle element of the list is calculated.
The value at the middle position is 5.
Since 7 is greater than 5, the algorithm **ignores the left half** and continues with the right half.

2. **Second iteration**
A new middle is calculated in the remaining right portion.
The value now checked is 8.
Since 7 is smaller than 8, the algorithm **moves left**.

3. **Third iteration**
The middle is recalculated again.
The value checked is 6.
Since 7 is greater than 6, the algorithm shifts right.

4. **Fourth iteration**
The middle position now contains 7.
The target is found, and the search stops.

Why the iteration count is only 4
Each step removes **half of the remaining elements**, drastically reducing the number of comparisons.
This is why binary search grows logarithmically and is much faster for large datasets.

11) A list contains 10 sorted elements. The target element is located at index 6. ***1 point***
Using the **linear search code from the case study**, how many iterations will be counted?

- 10

- 6
- 7
- 5

Yes, the answer is correct.
Score: 1

Accepted Answers:
7

12) In the binary search implementation, what primarily causes the number of iterations to reduce compared to linear search? ***1 point***

- The use of a flag variable
- The loop runs fewer times due to index jumps
- The list length is fixed
- Elements are compared sequentially

Yes, the answer is correct.
Score: 1

Accepted Answers:
The loop runs fewer times due to index jumps

13) If the searched element is **not present** in the list of 10 elements,which statement is true regarding the iteration count? ***1 point***

- Linear search will stop after half the list
- Binary search will always run exactly 10 times
- Binary search cannot handle missing elements
- Linear search will check all elements

Yes, the answer is correct.
Score: 1

Accepted Answers:
Linear search will check all elements

14) If the number of elements increases from 10 to 1,000, which graph best represents the growth of iterations for **binary search**? ***1 point***

- A slowly rising curve
- A flat horizontal line
- A zig-zag pattern
- A steep straight line

Yes, the answer is correct.
Score: 1

Accepted Answers:
A slowly rising curve

15) In the binary search code, what would happen if the list were **unsorted** but the same logic was applied? ***1 point***

- The algorithm would still work but slower
- The iteration count would double

- ○ Python would raise a runtime error
- ◉ The element may not be found even if present

Yes, the answer is correct.
Score: 1

Accepted Answers:
The element may not be found even if present